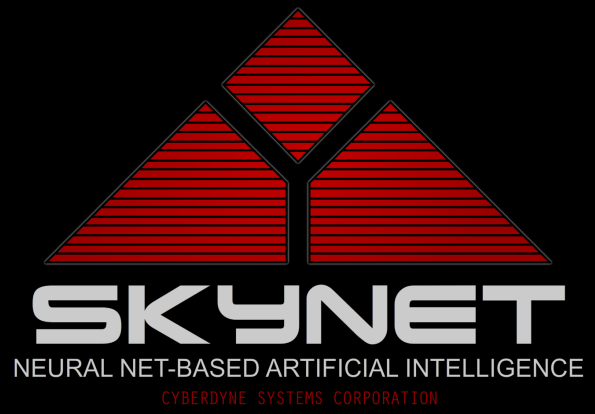


AUTONOMOUS QA PLATFORM

Auditoría web multiagente con revisión humana

Presentación preliminar para el equipo

15 de octubre de 2026 · Estado: prototipo funcional

Qué tenemos hoy

Un flujo completo: sitio + reglas → hallazgos con evidencia → revisión del equipo.

6
agentes especializados

8
categorías implementadas

22
hallazgos en la demo

62,5 s
duración observada

Ya implementado

Auditoría, revisión, actividad visible y configuración por archivos.

Próximo foco

Persistencia, Docker integral y ensayo con entradas desconocidas.

Corrida de práctica: 3 páginas válidas + 1 destino roto. No es un benchmark de calidad.

El objetivo: QA de contenido reutilizable

Detectar problemas de contenido sin depender de la marca ni del idioma del sitio de práctica.

Entrada

URL del sitio

Perfil lingüístico YAML

Referencias Markdown

Análisis

Reglas deterministas

Modelos con contexto

Validación de evidencias

Salida

Issues estructurados

Ubicación y corrección sugerida

Revisión humana

Alcance actual: calidad de contenido de sitios estáticos; sin corrección automática.

Entradas y contrato de salida

Perfil del sitio

Locale objetivo y de origen
Formatos de fecha, moneda y unidades
Voz de marca y reglas de estilo
Glosario: términos preferidos y prohibidos
Referencias vinculadas a cada página

Cada hallazgo incluye

Categoría y severidad
Página + tipo de elemento / selector
Descripción y evidencia
Resultado esperado + sugerencia
Confianza y estado de revisión

Los valores se leen de las entradas: cambiar de sitio no debe requerir cambiar código.

Stack: simple y explícito

CAPA	TECNOLOGÍA	RESPONSABILIDAD
Backend	Node.js 22 · TypeScript 5.8 · Express 5	API, ejecución y entrega del dashboard
Crawler	Fetch + Cheerio	Recorrer HTML y extraer texto, enlaces y medios
IA	OpenAI SDK 5 · Responses API	Salidas estructuradas e inspección de imágenes
Reglas / validación	YAML + Ajv / JSON Schema	Perfil configurable y contrato de issues
Frontend	HTML · CSS · JavaScript nativo	Actividad, filtros y revisión de hallazgos
Ejecución / pruebas	Docker Compose · node:test · tsx	Entorno reproducible y pruebas con mocks

Modelo observado: gpt-6-astra · configurable mediante OPENAI_MODEL. Estado almacenado en memoria.

DIAGRAMA

Arquitectura de extremo a extremo



Configuración cargada por el backend

Cliente compartido: estructura, reintentos y uso

Un proceso Node; workers lógicos concurrentes. No es una plataforma distribuida.

src/server.ts; src/orchestrator/audit.ts; src/agents/

PASO A PASO

Workflow de una auditoría

01 Configurar

URL, perfil y referencias

02 Descubrir

Crawl del mismo origen

03 Despachar

Workers por alcance

04 Comprobar

Reglas + modelos

05 Validar

Schema, evidencia, IDs

06 Revisar

Verify / Dismiss / Escalate

El estado del run y el estado de revisión de cada issue son conceptos distintos.

Qué decide el orquestador

Plan de ejecución

Registro: agente + categorías + alcance

Una tarea de Links para el sitio
Cinco tipos de agentes por página

Máximo 2 workers simultáneos

Control y resiliencia

Fallos aislados por worker

1 reintento ante errores transitorios
Timeout de modelo: 60 s por intento

Schema + deduplicación + estado previo

Demo actual: 6 agentes → 16 workers → 15 completados + 1 no aplicable.

src/orchestrator/audit.ts; src/agents/model-check.ts

Seis agentes, ocho categorías

AGENTE	CATEGORÍAS	CÓMO COMPRUEBA
Language	Ortografía y gramática	Modelo + citas exactas del texto
Links	Enlaces	Respuestas HTTP del mismo origen
Locale / Formatting	Formato + reglas locales	Fechas deterministas + reglas con modelo
Brand / Terminology	Voz de marca + terminología	Reglas de marca y glosario en contexto
Translation Quality	Calidad de traducción	Referencia Markdown vs. texto destino
Media	Imágenes y video	Recursos + alt + WebVTT + visión

Las comprobaciones visuales del modelo se identifican como observaciones que requieren revisión.

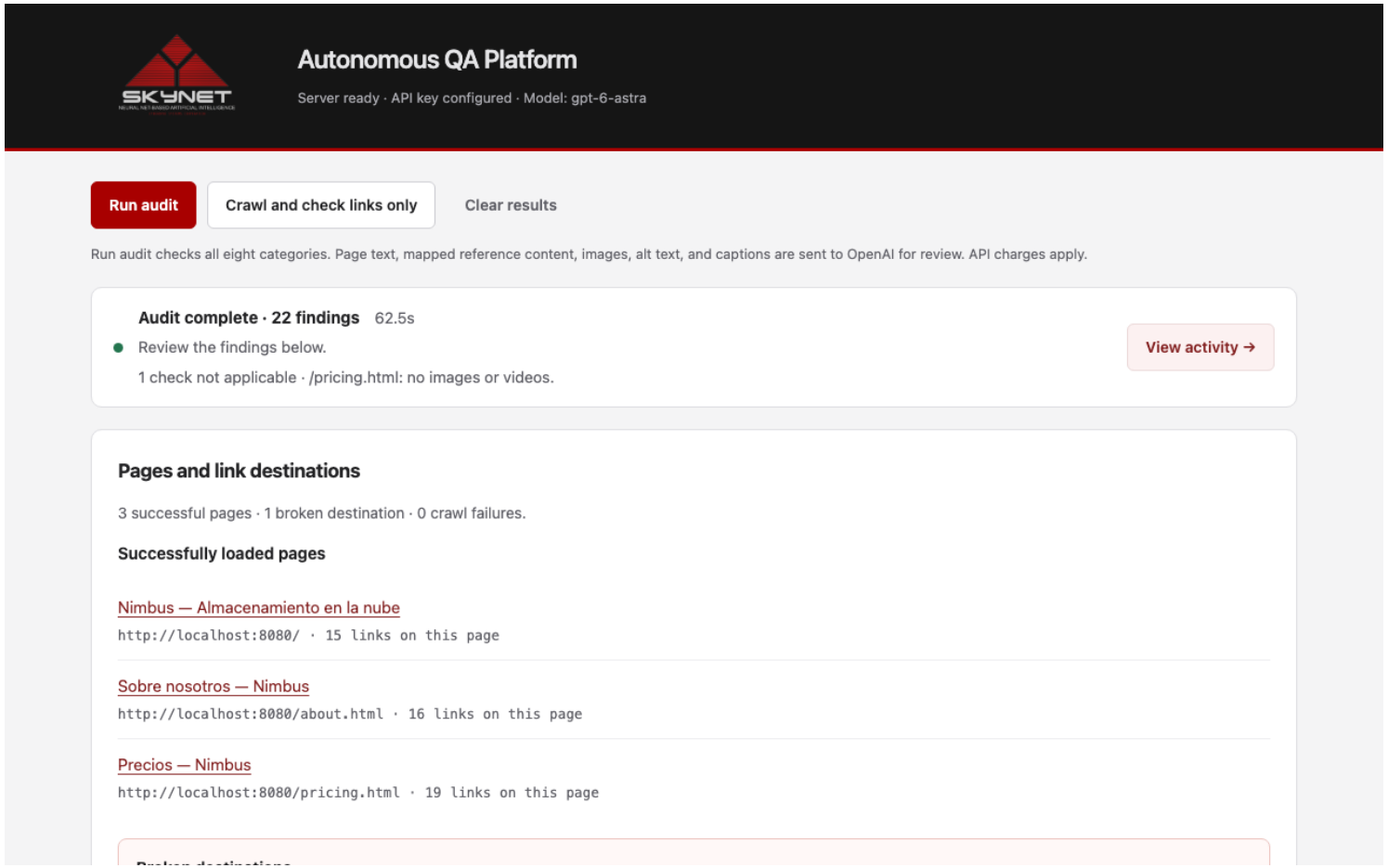
De una respuesta de IA a un hallazgo revisable

- Estructura validada: categoría, severidad, ubicación, evidencia y estado.
- Citas de texto verificadas contra las entradas; traducción valida origen y destino.
- Observaciones visuales explícitamente marcadas para revisión humana.
- IDs estables y deduplicación para evitar hallazgos repetidos.

Verified = hallazgo confirmado. No significa que el contenido del sitio ya esté corregido.

docs/ISSUE_SCHEMA.json; model-check.ts; translation-quality.ts; media.ts

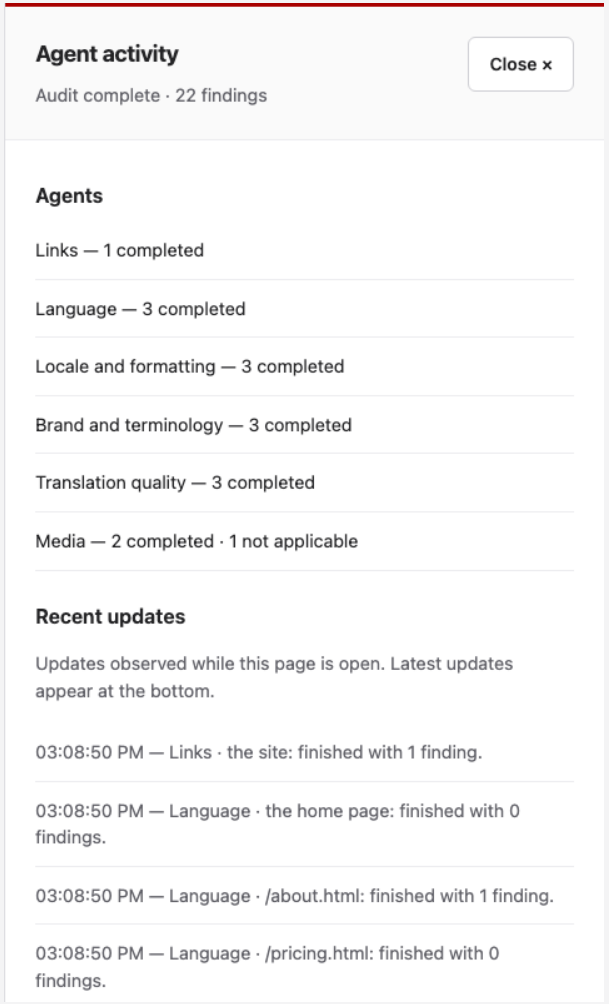
La app: una superficie de revisión



- Acciones de ejecución
- Resumen compacto
- Páginas y destinos rotos
- Filtros y hallazgos

Captura real del dashboard · localhost:3000

Actividad visible sin saturar la página



Vista humana

Quién está trabajando, qué terminó y qué requiere atención.

Detalles a demanda

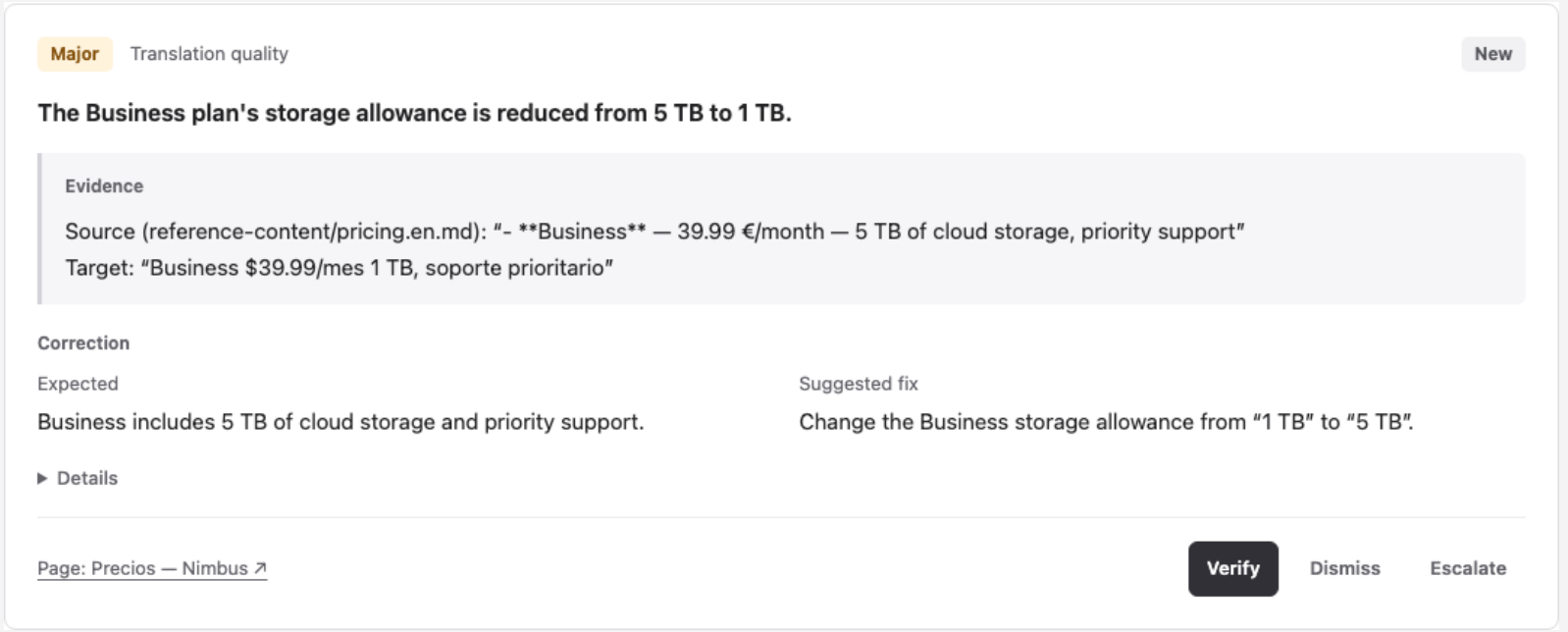
Cobertura, intentos, duración, tokens, costos y recursos.

“No aplicable” no es un fallo

Media en Pricing: no hay imágenes ni videos para inspeccionar.

activity.js; polling de GET /api/audits/latest

Hallazgos que el equipo puede revisar



Decisión humana

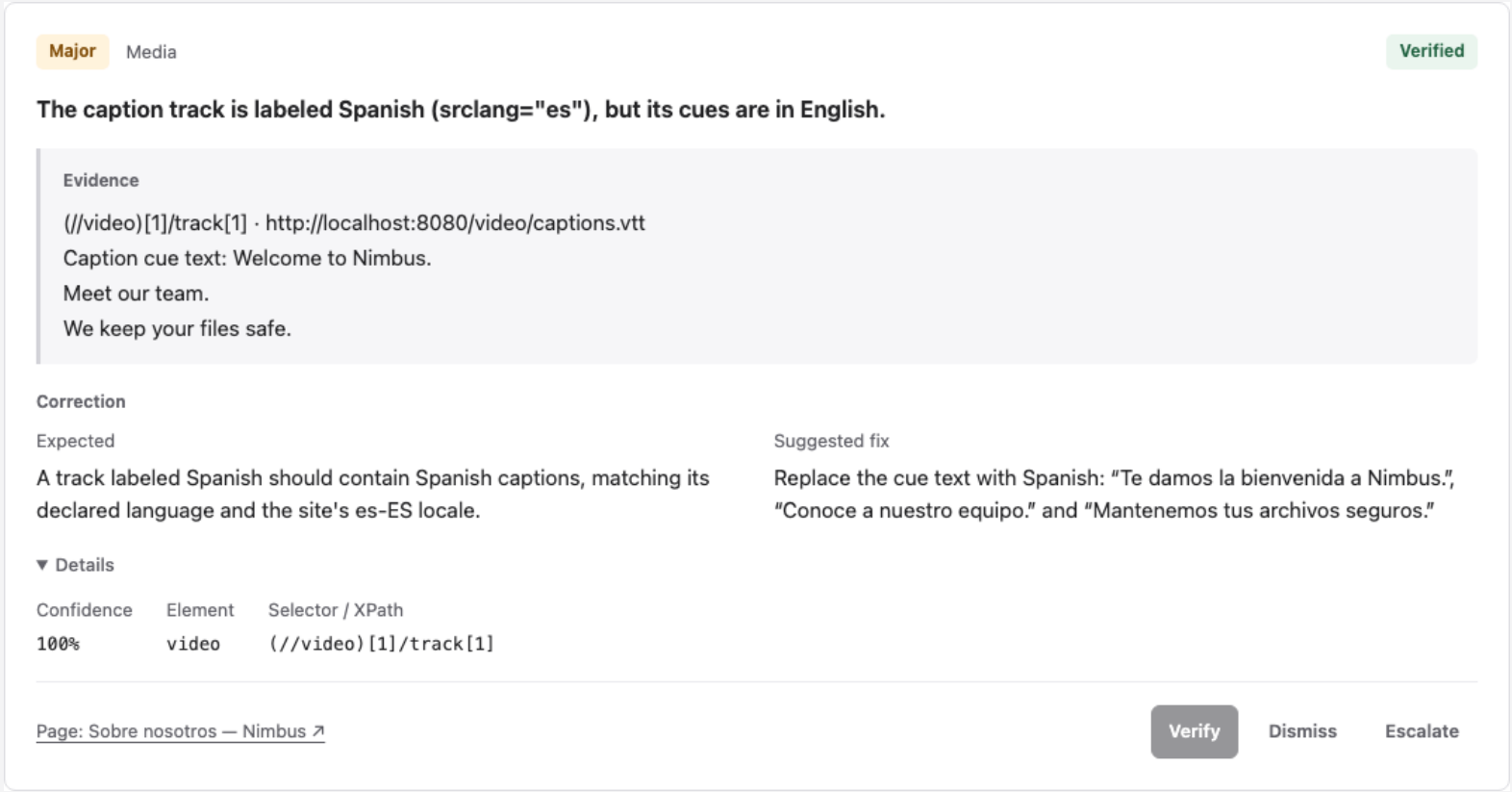
Verify
Confirmar el problema

Dismiss
Descartar el hallazgo

Escalate
Pedir revisión adicional

issues-view.js; captura real de Translation Quality

Caso real: subtítulos en el idioma equivocado



DÓNDE

About → Conoce al equipo

(//video)[1]/track[1]

/video/captions.vtt

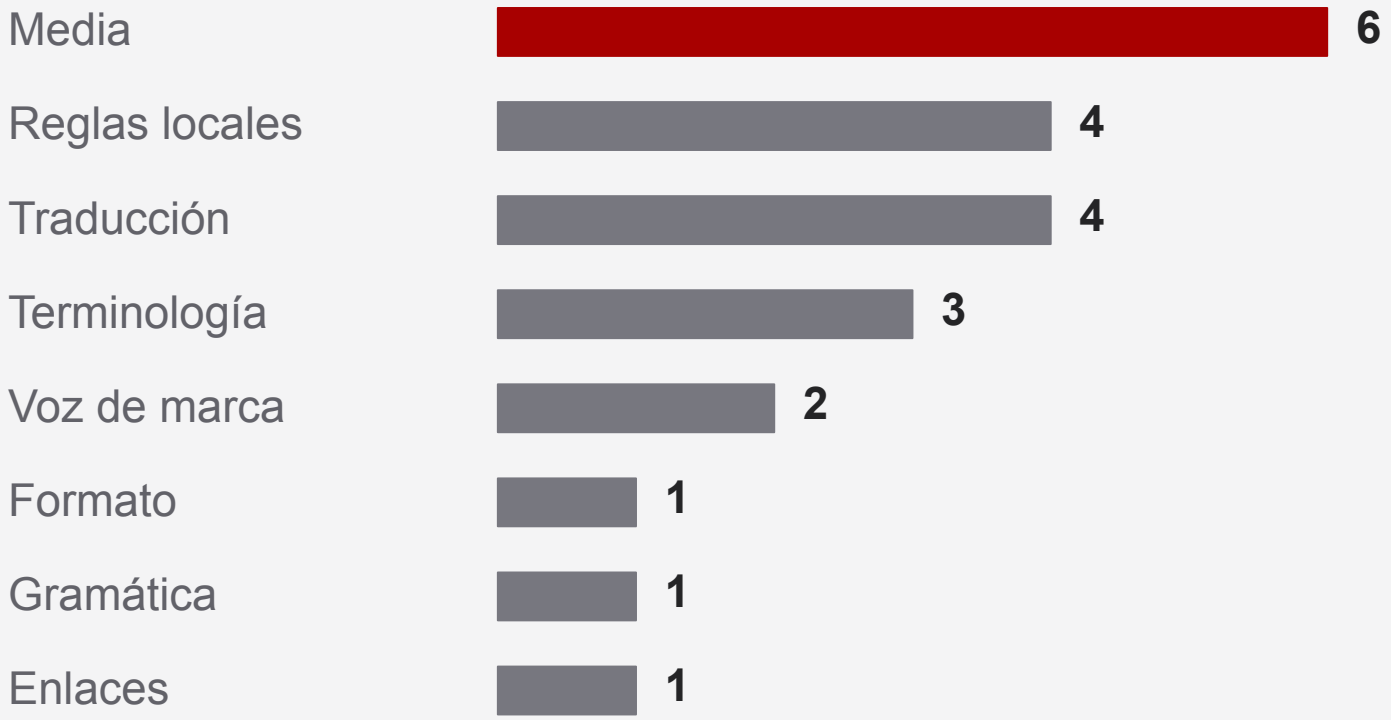
Etiqueta: Español
Contenido: inglés

about.html:68; video/captions.vtt; hallazgo media-e3c90b8d03179145

Resultado observado en el sitio de práctica

Una corrida completa

22 hallazgos
62,5 segundos
3 páginas válidas
1 destino roto
15 workers completados
1 check no aplicable



Smoke test: no demuestra precisión/recall ni garantiza el mismo resultado en otro sitio.

Qué validamos hasta ahora

57 tests aprobados

Contratos y evidencia
Errores y reintentos
Perfiles alternativos
Reset y estado de revisión
Cobertura / no aplicable

Navegador

Desktop y móvil
Teclado y foco
Panel de actividad
Tarjetas estables
Filtros y errores de guardado

Integración real

Corrida de 8 categorías
Evidencias inspeccionadas
Ejemplos de texto y medios

Pendiente: benchmark de calidad

Validación técnica ≠ exactitud semántica. Hace falta evaluación humana sistemática.

Cómo se ejecuta y configura

Desarrollo local

cd platform-template
npm install
npm run dev

Docker Compose

docker compose up --build
Configurado; validación integral pendiente.

Variables principales

SITE_URL
LINGUISTIC_ASSET_PATH
REFERENCE_CONTENT_PATH
ISSUE_SCHEMA_PATH
OPENAI_API_KEY / OPENAI_MODEL
Tarifas opcionales para estimar costos

API key en backend. Los inputs montados se leen; el contenido del sitio no se modifica.

platform-template/README.md; docker-compose.yml; .env.example

Límites actuales: qué no debemos prometer

Memoria y trazabilidad

Sin historial durable de runs ni auditoría persistente de revisiones.

Cobertura de enlaces

Faltan externos, destino en otro locale y coherencia texto/destino.

Escala y navegación

50 visitas; texto >24.000 caracteres sin chunking. No ejecuta JavaScript.

Medios y criterio del modelo

Sin reproducción/audio/sincronía de video; límites de recursos e imágenes.

Próximos pasos propuestos

ORDEN	ENTREGABLE	CRITERIO DE SALIDA
1	Persistencia e historial	Runs y revisiones sobreviven a reinicios
2	Validación Docker end to end	Arranque y auditoría reproducibles
3	Ampliar Links	Externos, locale y significado del destino
4	Ensayo con sitio desconocido	Sin cambios de código + revisión de calidad
5	Robustez y límites	Chunking y explicaciones consistentes

Prioridad para acordar hoy: consolidar la base antes de ampliar el alcance.

Guion para la demo y conversación del equipo

- Mostrar la corrida actual: 22 hallazgos y cobertura real.
- Abrir View activity: agentes, no aplicable y detalles técnicos.
- Abrir una tarjeta: evidencia, corrección y ubicación en el sitio.
- Explicar Verify / Dismiss / Escalate y Clear results.

Para acordar: prioridad de persistencia, responsables y criterio de calidad para el siguiente ensayo.

Anexo A · API disponible

ENDPOINT	USO
POST /api/audits	Inicia auditoría en segundo plano; devuelve 202
GET /api/audits/latest	Estado, workers, cobertura y hallazgos
POST /api/crawl	Crawl y Links sin llamadas al modelo
GET /api/pages · GET /api/issues	Consulta los resultados actuales
PATCH /api/issues/:id/status	verified / dismissed / escalated
DELETE /api/results	Borra resultados y decisiones de revisión
GET /api/health · POST /api/test-model	Estado del servidor y conexión con el modelo

Anexo B · Mapa del código y fuentes

RUTA	QUÉ ENCONTRAR
src/server.ts · src/config.ts	API, configuración y estado actual
src/crawler/crawl.ts	Descubrimiento y extracción HTML
src/orchestrator/audit.ts	Registro, despacho y concurrencia
src/agents/	Checks y cliente compartido de modelos
src/dashboard/public/	Interfaz, actividad y tarjetas de hallazgos
test/ · docs/ISSUE_SCHEMA.json	Pruebas y contrato de salida
platform-template/README.md · docs/HANDOFF.md	Operación, límites y próximos hitos

Rutas src/ y test/ relativas a platform-template/.